OOAD Lab 12

Name - Rithvik Rajesh Matta
Class - 6H
SRN - PES2UG23CS485

Output 1

```
PS C:\Users\rithv\sem6\OOAD\lab 12> javac .\DocumentProcessingSystem.java
PS C:\Users\rithv\sem6\OOAD\lab 12> java DocumentProcessingSystem
[LOG] Processing PDF file: file1.pdf
PDF file processed: file1.pdf
[LOG] Processing Word file: file2.docx
Word file processed: file2.docx
[LOG] Processing Excel file: file3.xlsx
Excel file processed: file3.xlsx
PS C:\Users\rithv\sem6\OOAD\lab 12>
```

Output 2

```
PS C:\Users\rithv\sem6\OOAD\lab 12> java SmartHomeSystem
Control Hub initialized.
[ControlHub] Managing: Basic Lighting
[ControlHub] Managing: Basic Security
[ControlHub] Managing: Basic Climate Control
[ControlHub] Managing: Basic Entertainment

--- Smart Home Configuration ---
Lighting: Basic LED Lights
Security: Standard Locks
Climate: Ceiling Fans
Entertainment: Basic TV
[ControlHub] Managing: Smart Lighting
[ControlHub] Managing: Advanced Security
[ControlHub] Managing: Advanced Climate Control
[ControlHub] Managing: Advanced Entertainment

--- Smart Home Configuration ---
Lighting: Smart Lighting System
Security: CCTV + Smart Locks
Climate: Smart Thermostat + AC
Entertainment: Home Theater System
PS C:\Users\rithv\sem6\OOAD\lab 12>
```

Code1

OOAD > lab 12 > DocumentProcessingSystem.java > DocumentProcessingSystem

```java
interface DocumentHandler {
    void processDocument(String fileName);
}

class PDFHandler implements DocumentHandler {
    @Override
    public void processDocument(String fileName) {
        Logger.getInstance().log("Processing PDF file: " + fileName);
        System.out.println("PDF file processed: " + fileName);
    }
}

class WordHandler implements DocumentHandler {
    @Override
    public void processDocument(String fileName) {
        Logger.getInstance().log("Processing Word file: " + fileName);
        System.out.println("Word file processed: " + fileName);
    }
}

class ExcelHandler implements DocumentHandler {
    @Override
    public void processDocument(String fileName) {
        Logger.getInstance().log("Processing Excel file: " + fileName);
        System.out.println("Excel file processed: " + fileName);
    }
}

class DocumentFactory {
    public static DocumentHandler getDocumentHandler(String type) {
        if (type == null) {
            throw new IllegalArgumentException("Type cannot be null");
        }

        switch (type.toLowerCase()) {
            case "pdf":
                return new PDFHandler();
            case "word":
                return new WordHandler();
            case "excel":
                return new ExcelHandler();
            default:
                throw new IllegalArgumentException("Unknown document type: " + type);
        }
    }
}

class Logger {
    private static Logger instance;

    private Logger() {}

    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
```

```java

class Logger {
    private static Logger instance;

    private Logger() {}

    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}

public class DocumentProcessingSystem {
    Run | Debug
    public static void main(String[] args) {

        DocumentHandler pdf = DocumentFactory.getDocumentHandler("pdf");
        pdf.processDocument("file1.pdf");

        DocumentHandler word = DocumentFactory.getDocumentHandler("word");
        word.processDocument("file2.docx");

        DocumentHandler excel = DocumentFactory.getDocumentHandler("excel");
        excel.processDocument("file3.xlsx");
    }
}
```

Code 2

OOAD > lab 12 > SmartHomeSystem.java > SmartHomeSystem

```java
class ControlHub {
    private static ControlHub instance;

    private ControlHub() {
        System.out.println("Control Hub initialized.");
    }

    public static synchronized ControlHub getInstance() {
        if (instance == null) {
            instance = new ControlHub();
        }
        return instance;
    }

    public void controlDevice(String device) {
        System.out.println("[ControlHub] Managing: " + device);
    }
}

class SmartHome {
    private String lighting;
    private String security;
    private String climate;
    private String entertainment;

    public void setLighting(String lighting) {
        this.lighting = lighting;
    }

    public void setSecurity(String security) {
        this.security = security;
    }

    public void setClimate(String climate) {
        this.climate = climate;
    }

    public void setEntertainment(String entertainment) {
        this.entertainment = entertainment;
    }

    public void showConfiguration() {
        System.out.println("\n--- Smart Home Configuration ---");
        System.out.println("Lighting: " + lighting);
        System.out.println("Security: " + security);
        System.out.println("Climate: " + climate);
        System.out.println("Entertainment: " + entertainment);
    }
}

abstract class SmartHomeBuilder {
    protected SmartHome home;
    protected ControlHub hub = ControlHub.getInstance(); // Singleton used here

    public SmartHomeBuilder() {
        home = new SmartHome();
    }

    public abstract void buildLighting();
    public abstract void buildSecurity();
    public abstract void buildClimate();
```

OOAD > lab 12 > SmartHomeSystem.java > SmartHomeSystem

```java
abstract class SmartHomeBuilder {
    public abstract void buildClimate();
    public abstract void buildEntertainment();

    public SmartHome getHome() {
        return home;
    }
}

class BasicHomeBuilder extends SmartHomeBuilder {

    @Override
    public void buildLighting() {
        home.setLighting("Basic LED Lights");
        hub.controlDevice("Basic Lighting");
    }

    @Override
    public void buildSecurity() {
        home.setSecurity("Standard Locks");
        hub.controlDevice("Basic Security");
    }

    @Override
    public void buildClimate() {
        home.setClimate("Ceiling Fans");
        hub.controlDevice("Basic Climate Control");
    }

    @Override
    public void buildEntertainment() {
        home.setEntertainment("Basic TV");
        hub.controlDevice("Basic Entertainment");
    }
}

class AdvancedHomeBuilder extends SmartHomeBuilder {

    @Override
    public void buildLighting() {
        home.setLighting("Smart Lighting System");
        hub.controlDevice("Smart Lighting");
    }

    @Override
    public void buildSecurity() {
        home.setSecurity("CCTV + Smart Locks");
        hub.controlDevice("Advanced Security");
    }

    @Override
    public void buildClimate() {
        home.setClimate("Smart Thermostat + AC");
        hub.controlDevice("Advanced Climate Control");
    }

    @Override
    public void buildEntertainment() {
        home.setEntertainment("Home Theater System");
        hub.controlDevice("Advanced Entertainment");
    }
```

```java
        hub.controlDevice("Advanced Security");
    }

    @Override
    public void buildClimate() {
        home.setClimate("Smart Thermostat + AC");
        hub.controlDevice("Advanced Climate Control");
    }

    @Override
    public void buildEntertainment() {
        home.setEntertainment("Home Theater System");
        hub.controlDevice("Advanced Entertainment");
    }
}

class HomeDirector {
    public void constructHome(SmartHomeBuilder builder) {
        builder.buildLighting();
        builder.buildSecurity();
        builder.buildClimate();
        builder.buildEntertainment();
    }
}

public class SmartHomeSystem {
    Run | Debug
    public static void main(String[] args) {

        HomeDirector director = new HomeDirector();

        SmartHomeBuilder basicBuilder = new BasicHomeBuilder();
        director.constructHome(basicBuilder);
        SmartHome basicHome = basicBuilder.getHome();
        basicHome.showConfiguration();

        SmartHomeBuilder advancedBuilder = new AdvancedHomeBuilder();
        director.constructHome(advancedBuilder);
        SmartHome advancedHome = advancedBuilder.getHome();
        advancedHome.showConfiguration();
    }
}
```